

Architecture du modèle, amélioration et détection du déséquilibre

Comprendre construire_modele(), le corriger, et savoir où regarde-t-on pour voir un déséquilibre entre classes

Ce document explique le fonctionnement de l'architecture actuelle du réseau, signale un bug présent dans le code, propose des pistes concrètes d'amélioration, puis détaille les métriques à utiliser pour repérer un déséquilibre entre classes et savoir à quel niveau il se situe.

1. Comment fonctionne le modèle actuel

1.1 L'entrée

```
entree = keras.Input(shape=(5,), name="profil")
```

Le réseau attend un vecteur de 5 nombres par exemple : les 5 réponses du questionnaire (chacune valant 1, 2, 3 ou 4). C'est la porte d'entrée du réseau, aucune couche de calcul n'y est encore appliquée.

1.2 Le tronc commun (couches Dense)

Le modèle empile 4 couches Dense de 128 neurones, chacune avec une activation ReLU. C'est le "tronc commun" : une représentation interne partagée, apprise à partir des 5 réponses, et utilisée ensuite par toutes les têtes de sortie.

Élément	Rôle
Dense(128)	Combine les 5 entrées en 128 nouvelles valeurs (features apprises)
activation="relu"	Introduit de la non-linéarité : sans elle, empiler des couches reviendrait à une seule transformation
4 couches empilées	Permet au réseau d'apprendre des combinaisons de plus en plus abstraites des réponses du p

1.3 Les 63 têtes de sortie

```
sorties = [keras.layers.Dense(4, activation="softmax", name=f"reco_{i}")(x) for i in range(NB_RECOS)]
```

À partir de la même représentation partagée x , le modèle branche 63 petites couches de sortie indépendantes (une par recommandation). Chacune a 4 neurones avec une activation softmax, qui transforme les sorties en probabilités sommant à 1 sur les 4 classes (Absente, Moyenne, Elevee, Critique).

C'est une architecture multi-tâches (multi-output) : un seul tronc commun, mais 63 classifications indépendantes à 4 classes chacune, entraînées simultanément.

1.4 La compilation

```
modele.compile(optimizer="adam", loss=["sparse_categorical_crossentropy"] * NB_RECOS, metrics=[["accuracy"]] * NB_RECOS)
```

Élément	Rôle
optimizer="adam"	Algorithme qui ajuste les poids du réseau à chaque batch, en combinant momentum et taux d'ap
sparse_categorical_crossentropy	Fonction de perte adaptée quand les classes sont des entiers (0,1,2,3) plutôt que des vecteurs c
Une loss par tête	Chacune des 63 sorties a sa propre perte ; Keras additionne les 63 pertes pour obtenir la perte t
metrics=[["accuracy"]]*NB_RECOS	Accuracy calculée séparément pour chacune des 63 têtes de sortie

2. Bug détecté : le Dropout n'est pas connecté

Dans le code fourni, chaque appel à Dropout crée bien une couche, mais son résultat n'est jamais réutilisé :

```
x = keras.layers.Dense(128, activation="relu")(entree)
keras.layers.Dropout(0.1)(x)    # <- résultat perdu, jamais réassigné
x = keras.layers.Dense(128, activation="relu")(x)    # <- repart du x d'avant, sans dropout
```

Le Dropout est bien créé et appliqué à x pour produire une nouvelle sortie, mais cette sortie n'est stockée dans aucune variable. La couche Dense suivante repart donc du x d'origine (sans dropout), et le Dropout est complètement absent du graphe de calcul : il n'a aucun effet régularisant, comme s'il n'existait pas.

Version corrigée :

```
x = keras.layers.Dense(128, activation="relu")(entree)
x = keras.layers.Dropout(0.1)(x)    # <- réassigné à x
x = keras.layers.Dense(128, activation="relu")(x)
x = keras.layers.Dropout(0.1)(x)
...
```

Il faut systématiquement réaffecter le résultat de chaque couche à x (ou à une nouvelle variable) pour qu'elle soit réellement intégrée dans le flux du réseau.

3. Pistes concrètes d'amélioration

3.1 Corriger et ajuster le Dropout

Une fois corrigé (section 2), un Dropout de 0.2 à 0.3 est un point de départ raisonnable pour ~1800 exemples. Un Dropout trop faible (0.1) régularise peu ; trop fort (>0.5) peut empêcher le modèle d'apprendre correctement.

3.2 Régularisation L2

```
keras.layers.Dense(128, activation="relu", kernel_regularizer=keras.regularizers.l2(0.001))
```

Pénalise les poids trop grands pendant l'entraînement, ce qui limite la complexité du modèle et réduit le risque de surapprentissage, en complément (ou à la place) du Dropout.

3.3 Normalisation par batch (BatchNormalization)

Ajoutée après une couche Dense (avant ou après l'activation selon les préférences), elle stabilise et accélère l'entraînement en normalisant les activations à chaque batch.

3.4 Taille et profondeur du réseau

4 couches de 128 neurones est déjà une architecture assez large pour seulement 5 entrées et ~1800 exemples. Réduire à 2-3 couches (par exemple 128 → 64) peut suffire et réduire le risque de surapprentissage, tout en accélérant l'entraînement.

3.5 EarlyStopping et callbacks

Comme vu précédemment : EarlyStopping(monitor="val_loss", patience=15, restore_best_weights=True), passé dans callbacks=[...] lors du fit(), pour arrêter l'entraînement au bon moment automatiquement.

3.6 Pondération des classes (class_weight)

Si certaines classes de priorité sont beaucoup plus rares que d'autres (typiquement "Critique" ou "Elevee" par rapport à "Absente"), le modèle peut apprendre à les ignorer. Donner un poids plus élevé aux classes rares dans la fonction de perte force le modèle à leur accorder plus d'importance pendant l'entraînement.

4. Métriques pour détecter un déséquilibre entre classes

L'accuracy seule ne suffit pas : elle peut être élevée même si le modèle échoue complètement sur les classes rares (voir section précédente sur les guides déjà fournis). Voici les métriques à utiliser, et à quel niveau chacune permet de voir le déséquilibre.

4.1 Distribution des classes dans les données (niveau : dataset)

Avant même d'entraîner, compter le nombre d'exemples par classe (0=Absente, 1=Moyenne, 2=Elevée, 3=Critique) sur l'ensemble de Y aplati. Un déséquilibre se voit ici si une classe représente par exemple 90% des cas et une autre moins de 2%.

```
import numpy as np
valeurs, comptes = np.unique(Y.flatten(), return_counts=True)
for v, c in zip(valeurs, comptes): print(CLASSES[v], c)
```

4.2 Precision, Recall, F1-score par classe (niveau : par classe)

Métrique	Ce qu'elle révèle	Déséquilibre visible si...
Precision	Parmi les prédictions "Critique", combien sont réellement critiques	Beaucoup de faux positifs sur une classe rare
Recall (rappel)	Parmi les vraies "Critique", combien sont retrouvés	Le modèle rate systématiquement une classe rare
F1-score	Moyenne harmonique de precision et recall	Score bas = la classe est mal gérée dans les deux sens

`sklearn.metrics.classification_report(y_true, y_pred, target_names=CLASSES)` affiche ces trois métriques pour chacune des 4 classes, avec la colonne "support" qui indique le nombre d'exemples réels de chaque classe — un déséquilibre apparaît clairement si le "support" d'une classe est très faible par rapport aux autres.

4.3 Matrice de confusion (niveau : entre paires de classes)

Montre précisément quelles classes sont confondues entre elles (ex : "Elevée" prédit à la place de "Critique"). Un déséquilibre se traduit souvent par une colonne ou une ligne presque vide pour la classe rare : le modèle ne la prédit presque jamais, ou ne la retrouve presque jamais.

4.4 Macro-average vs Weighted-average (niveau : global, mais sensible au déséquilibre)

Moyenne	Calcul	Ce qu'elle montre
Macro-average	Moyenne simple des scores par classe, chaque classe compte pareil	Une classe importante est mal prédite, même si elle a peu d'exemples
Weighted-average	Moyenne pondérée par le nombre d'exemples de chaque classe	Peu importe même si la classe rare est mal prédite (d)

Un grand écart entre macro-average et weighted-average est un signal clair de déséquilibre : cela signifie que le modèle est bon sur la classe majoritaire mais faible sur les classes minoritaires.

4.5 Accuracy sur les recommandations réellement actives (niveau : par profil)

Déjà présente dans le code de test : en excluant la classe "Absente" (largement majoritaire), cette métrique isole la performance du modèle sur les cas qui comptent réellement, profil par profil.

En résumé : la distribution des classes révèle le déséquilibre dans les données ; precision/recall/F1 et la matrice de confusion révèlent où (quelle classe, quelle confusion) le modèle échoue ; et la comparaison macro/weighted révèle si ce déséquilibre impacte réellement la performance globale perçue.